

UNIT- V :

→ Replication :

- Introduction
- System model & group communication
- Fault tolerant services
- Transactions with replicated data.

→ Distributed shared memory:

- Design & Implementation issues
- Consistency models

* Replication Replica-copy ⇒ Same data is stored in many systems
- so always available

- It is the process of maintaining multiple copies (replicas) of data or services on different machines in a ds to improve availability, reliability, performance
- Data is stored at multiple locations
- Each copy is called a replica
- Ensures availability even during failures
- Used in :
 - cloud
 - web
 - databases

Types — ② → Data duplication

①. Active Replication

- All replicas process the same request
- All produce same result

②. Passive Replication (Primary-Backup)

- One primary server handles requests
- Backup servers updated.

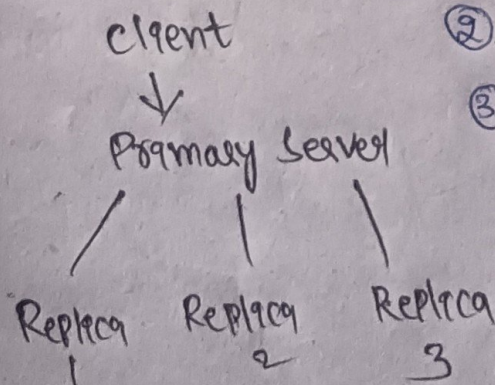
⇒ easy than active Replication

Steps

①. client req to server

②. Server processes request

③. Updates sent to replicas



⇒ High availability

- Data available even if server fails

⇒ Fault Tolerance

- System continues working during failures

⇒ Improved Performance

- Faster access ~~due to~~ ~~no~~

⇒ Load Balancing

- Requests shared among multiple servers

⇒ Scalable

- Handles large no. of users

Need,
Characteristics
Advantages

Strategies ⇒

- Edge replication (immediate update)
- Lazy replication (delayed update)

Disadvantages

- Complex
- High storage
- Storage cost high

eg: ① Google stores same data in multiple servers

Replication = storing same data in multiple places for reliability

eg:

②. Banking Systems

⇒ Your bank data is stored in multiple servers

⇒ ATM + mobile + branch ⇒ Same balance

③. WhatsApp

④. Netflix, Youtube

⑤. Amazon, Flipkart

* System Model & Group Communication

* System Model

- A System Model describes the structure, components & behav
 - & behaviour of processes, commⁿ in a ds
- ⇒ It explains how computers in ds work & communicate
- defines how processes interact
- explain how they communicate
- Identifies failure conditions
- used to analyze performance & reliability.

Types of System Models - (3) - E

①. Physical Model

- Shows actual h/w (computers, n/w)
- ex: machines connected via n/w

②. Architectural Model (client-server, P2P, Hybrid, N-Tier)

③. Fundamental Model

- Interaction model
- Failure model
- Security model

Same as

1st unit

Group Communication

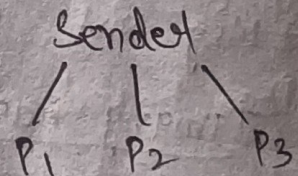
- It is a mechanism (group members)
- where a msg is sent to multiple processes in a ds
- Sending one msg to many systems at once
- used when multiple nodes need same data
- efficient than sending individually
- Supports distributed applications
- used in replication & synchronization
- Improves Performance

Types of Commⁿ

- ①. Unicast - one sender → one receiver
- ②. Multicast - one sender → selected group
- ③. Broadcast - one sender → all nodes

- Fastest
- Fault tolerance
- Scalability

→ Group mgmt



(Ex) ①. online meeting Zoom

②. Live Streaming

(dis)

↳ Complex

↳ msg loss

* Fault Tolerant Services

- It is a service.
- It continues to operate/work correctly even in the presence of faults.
- Even some components fail, the system works.
- Failures are common in ds.
- Fault tolerance ensures continuous service.
- Uses techniques replication & recovery.
- Is important for critical applications.

Types of Failures

1. Crash Failure → System stops working completely

- Process stops & does not recover

2. Omission Failure

- Msgs are lost
- Msg not sent or received

3. Timing Failure

- Response delayed

Techniques

1. Replication

2. Redundancy (extra components added, backup)

3. Failure Detection

4. Recovery

5. Checkpointing

Steps

- ⇒ Failure occurs
- ⇒ System detects failure
- ⇒ Backup or replica activated
- ⇒ System continues operation
- ⇒ Recovery Process starts

Client



Primary Server (fails)



Backup Server (^{Activate} ~~fail~~)

Adv

- Reliability
- Availability
- Automatic recovery
- Continuous operation

Dis

- High cost
- complex implementation

ex Banking System
• Google cloud

* DSM - Distributed Shared Memory ^{→ fake shared memory}

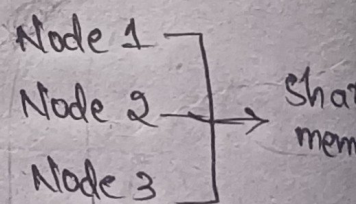
- DSM is a system
- Many computers share memory as if it's one single memory.
 - even though it is actually in different places.
- Different computers use one common memory (virtually)
 - Memory is physically separate
 - Appears as one shared memory
 - Uses read/write operations
 - No need of msg passing
 - System hides complexity
 - Used in ds
 - Supports parallel processing.

Types - ②

①. Page-based (data in pages)

②. Object-based (data as objects)

- Transparency
- distributed memory
- Supports multiple users



Adv

- Faster
- Good for Parallel Systems

Dis

- Complex
- Data inconsistency
- Slower than local memory
- Difficult to maintain

Ex: ①. Google Docs

②. Multiplayer games

Real-world Use

- Cloud Systems
- Parallel computing
- Cluster System

* Consistency Models

- A consistency model defines the rules
 - that specify when updates made by one process become visible to other processes in a ds
- ⇒ When & how changes appear to other systems.
- ⇒ Ensures correct data view

Types — ④ [S-S-C-E]

↑ — high
↓ — low

1. Strict consistency

- All processes see same value instantly.
- Latency ↑^{High}, Scalability ↓^{low}, Fault Tolerance ↓
- ~~Very~~ Strong consistent, Strongest model

2. Sequential consistency [All processes see same order of operations.]

- Visibility after ordering.
- Strong consistency.
- High accuracy, medium latency, medium fault tolerance

3. Causal consistency

- Only related operations follow order.

4. Eventual consistency

- Updates spread slowly.

● COMPARISON TABLE (VERY IMPORTANT FOR EXAM)

Feature	Strict Consistency	Sequential Consistency	Causal Consistency	Eventual Consistency
Definition	All processes see latest value immediately	All processes see operations in same order	Only causally related operations are ordered	Updates become consistent over time
Time Requirement	Real-time (instant)	No real-time requirement	Partial ordering	No time guarantee
Order Guarantee	Absolute order	Same order for all	Cause-effect order only	No order guarantee
Consistency Level	Very High	High	Medium	Low
Performance	Very Low	Medium	High	Very High
Implementation	Very difficult	Moderate	Complex	Easy
Practical Usage	Not practical	Used in systems	Used in distributed apps	Widely used
Example	Ideal memory system	Multi-thread programs	Chat apps (message order)	DNS, WhatsApp

Feature	Strict	Sequential	Causal	Eventual
Immediate visibility	YES	NO	NO	NO
Same order for all	YES	YES	PARTIAL	NO
Cause-effect order	YES	YES	YES	NO
Uses real-time	YES	NO	NO	NO
Allows delay	NO	YES	YES	YES
Consistency strength	HIGH	HIGH	MEDIUM	LOW
Complexity	HIGH	MEDIUM	MEDIUM	LOW
Practical usage	NO	LIMITED	YES	YES
Data may differ temporarily	NO	NO	YES	YES
Used in real systems	NO	SOME	YES	YES

Feature	Strict	Sequential	Causal	Eventual
Speed	Low	Medium	High	Very High
Scalability	Low	Medium	High	Very High
Performance	Low	Medium	High	Very High
Accuracy	Very High	High	Medium	Low
Complexity	Very High	Medium	Medium	Low
Practical Use	No	Limited	Yes	Yes (most used)
Data Freshness	Always latest	Ordered	Depends	Delayed
Synchronization	Very High	High	Medium	Low
Implementation	Very Difficult	Moderate	Complex	Easy
Real-world Usage	No	Some	Many apps	Cloud systems

